□□ Class 1: Smart Contract Foundations

□ 1. Introduction to Smart Contracts

□ What is a Smart Contract?
A smart contract is a **self-executing digital agreement**. The terms of the contract are written directly into code and stored on a blockchain (like Ethereum). It runs automatically when predetermined conditions are met.

□ The "Trustless Digital Vending Machine" Analogy:

- **Insert Money:** You send ETH to the contract.
- **Select Item:** You call a function on the contract.
- **Automatic Delivery:** The contract automatically delivers the digital asset or service.
- **No Human Intervention:** No cashier is needed; you trust the machine's programming.

□ Key Features:

- □ **Immutable:** Once deployed, the code cannot be altered.
- □ **Transparent:** The code is visible to anyone on the blockchain.
- □ **Autonomous:** Executes its logic automatically.
- □ **Trustless:** No need to trust a middleman; the code enforces the rules.
- □ **Handles Digital Assets:** Can send, receive, and manage cryptocurrencies like ETH and other tokens.

□ 1.1. Real-World Applications of Smart Contracts

□ DeFi (Decentralized Finance)

- □ **Lending & Borrowing:** Platforms like Aave and Compound use smart contracts to allow users to lend their crypto and earn interest or borrow against their holdings without a bank.
- □ **Decentralized Exchanges (DEXs):** Uniswap uses smart contracts to enable peer-to-peer trading of tokens without a central order book.
- □ **Yield Farming & Liquidity Pools:** Users can "stake" their tokens in a smart contract to provide liquidity for a trading pair and earn rewards.

 NFTs (Non-Fungible Tokens)

- **Digital Art & Collectibles:** Smart contracts (like ERC-721) power NFTs, proving unique ownership of a digital item.
- **NFT Marketplaces:** Platforms like OpenSea are built on smart contracts that facilitate the minting, listing, and trading of NFTs.
- **Royalty Enforcement:** Creators can program royalties into the NFT's smart contract, ensuring they get a percentage of all future sales automatically.

 DAOs (Decentralized Autonomous Organizations)

- **On-Chain Voting:** Members holding governance tokens can vote on proposals directly through a smart contract.
- **Treasury Management:** The organization's funds are held in a smart contract and can only be spent based on the outcome of member votes.
- **Collective Governance:** Rules for membership, proposals, and voting are encoded in smart contracts, eliminating the need for a central board.

 Supply Chain

- **Track & Trace:** A product's journey from manufacturer to consumer can be recorded on the blockchain at each step.
- **Proof of Authenticity:** Each step can be verified with a unique hash, making it nearly impossible to counterfeit goods.

 E-Voting

- **Secure Registration:** Voter identities can be registered on-chain.
- ✔ **Tamper-Proof Ballots:** Once a vote is cast, it is immutably recorded.
- **Instant & Transparent Tallying:** Results can be calculated automatically and are open for audit.

 Insurance

- ☀ **Parametric Insurance:** A crop insurance smart contract can be programmed to automatically pay out if data from a weather oracle shows rainfall below a certain level.
- **Catastrophe Bonds:** Payouts for events like earthquakes can be triggered by verified seismic data.
- **Instant Settlement:** Eliminates lengthy claims processes.

1.2. How Smart Contracts Differ from Traditional Programs

Feature	Traditional Program	Smart Contract (Blockchain)
Execution Environment	Centralized server or cloud	Decentralized blockchain (EVM)
Upgradability	Easy to update and patch	Immutable once deployed (requires new contract)
Trust Model	Requires trusting the company/server	Trustless; rules are enforced by code & consensus
Security	Relies on firewalls, passwords, etc.	Secured by cryptographic proofs and network consensus
Auditability	Code is often private and proprietary	Code is fully transparent and verifiable by all
Money Handling	Requires integration with banks/payment processors	Can natively hold and transfer crypto assets (ETH)
Risk of Downtime	High (single point of failure)	Very low (distributed across thousands of nodes)

Summary & Key Takeaways

- Smart contracts are the **backbone of Web3**, enabling automated, transparent, and decentralized applications (dApps).
- They are used across **finance, art, governance, logistics, and more**.
- Learning about them opens doors to building **dApps, DeFi platforms, NFT projects, and DAOs**.
- They differ fundamentally from traditional programs in execution, trust, and immutability.

 Real Example in Daily Life:
Imagine a **smart contract for a bet** on a football match:

1. You and your friend each send 0.1 ETH to the contract.
2. The contract checks the official match result via a trusted data source (an **oracle**).
3. The winner automatically receives the entire 0.2 ETH.
   No need to trust a referee or worry about the other person not paying.

---

2. Solidity Programming Language Basics

 What is Solidity?
Solidity is a **high-level, contract-oriented, statically typed programming language** designed for writing smart contracts that run on the Ethereum Virtual Machine (EVM). Its syntax is influenced by JavaScript, Python, and C++.

 Key Concepts (Without Code):

- **Pragma and Versioning:** The `pragma` directive tells the compiler which version of Solidity to use, ensuring code compatibility. For example, `^0.8.0` means "use version 0.8.0 or any newer version that does not break functionality."
- **Structure of a Smart Contract:** A contract is like a **class** in object-oriented programming. It contains:
  - **State Variables:** Permanently stored on the blockchain (e.g., a `balance`).
  - **Constructor:** A special function that runs once during deployment to initialize the contract.
  - **Functions:** Define the actions and logic of the contract.
- **Comments:** Used to explain code for human readers. Ignored by the compiler (`//` for single-line, `/* ... */` for multi-line).

 State Variables vs Local Variables

1 State Variable

- □ **Definition:** A variable declared outside of any function. Its value is stored **permanently on the blockchain**.
- □ **Explanation:**
  - Lifetime is the same as the contract itself.
  - Used to store critical data like balances, ownership, and totals.
  - **Writing to a state variable costs gas.**
- □ **Analogy:** A **whiteboard in a classroom**—whatever is written stays until it's explicitly erased, and everyone can see it.

2□□ Local Variable

- □ **Definition:** A variable declared **inside a function**. It exists only while that function is executing.
- □ **Explanation:**
  - Temporary and is destroyed after the function finishes.
  - Used for intermediate calculations.
  - **Does not cost gas to store.**
- □ **Analogy:** A **scratch piece of paper** you use during a meeting—you use it for notes and throw it away afterward.

□ Comparison Table

Feature	□ State Variable	□ Local Variable
Declaration	Outside functions	Inside a function
Lifetime	Permanent (on blockchain)	Temporary (during function execution)
Storage Location	Blockchain Storage	Memory
Gas Cost	High cost to write	No storage cost
Accessibility	Can be accessed by other functions	Only accessible within its function

---

□ Basic Data Types & Keywords

Solidity provides data types optimized for the blockchain.

Type	Description	Example Use Case
`uint`	Positive integers (including zero)	Storing a user's age, token balance, or count.
`int`	Both positive and negative integers	Tracking temperature or profit/loss.
`bool`	Boolean (`true` or `false`)	Toggles, on/off switches, access control flags.
`string`	Sequence of characters (text)	Storing a name, description, or message.
`address`	Holds an Ethereum wallet address (20 bytes)	Storing the contract owner's address or a user's wallet.
`bytes32`	Fixed-size binary data	Storing hashes (e.g., for file verification).
`constant`	Value fixed at compile time and cannot change	Defining a maximum supply that is hardcoded.
`immutable`	Value set once during deployment and then fixed	Storing the deployer's address, which is known only at deployment.

□ Operators in Solidity

Operators perform actions on variables and values.

□ 1. Arithmetic Operators

- `+` (Addition), `-` (Subtraction), `*` (Multiplication), `/` (Division), `%` (Modulo - remainder).

- **Important:** Solidity does not natively support decimals for these operations; it uses integer math.

2. Comparison Operators

- `==` (Equal to), `!=` (Not equal to), `<` (Less than), `>` (Greater than), `<=` (Less than or equal), `>=` (Greater than or equal).
- Return a `bool` value (`true` or `false`).

3. Logical Operators

- `&&` (AND): `true` only if **both** conditions are true.
- `||` (OR): `true` if **at least one** condition is true.
- `!` (NOT): Reverses the boolean value.

4. Assignment Operators

- `=` (Assign), `+=` (Add and assign), `-=` (Subtract and assign), `*=` (Multiply and assign), `/=` (Divide and assign).

Solidity Data Structures

Data structures help you organize and manage data efficiently.

Data Structure	Description	Real-World Analogy & Use Case
Array	An ordered list of items.	A **row of lockers**—each has a number (index) and can hold an item. Use case: A list of scores.
Struct	A custom type that groups related variables.	A **form** with fields (name, age, ID). Use case: Defining a `User` profile.
Mapping	A key-value	A **phonebook**—you look up a name (key) to find a

Data Structure	Description	Real-World Analogy & Use Case
	store (like a dictionary).	number (value). Use case: `address -> balance`.
Nested Mapping	A mapping within another mapping.	A **company's yearly ledger**—`(Employee -> (Year -> Salary))`.
Array of Structs	A list of structured items.	A **class roster**—a list where each entry is a student's form.
Mapping of Structs	A key that points to a structured object.	A **product catalog**—a product ID (key) points to a product's details (struct).
Enum	A custom type with a fixed list of possible values.	A **dropdown menu** with options like `Status.Pending`, `Status.Approved`, `Status.Rejected`.

Loops in Solidity

Loops execute a block of code repeatedly.

- `for` **loop:** Used when you know how many times you want to iterate.
- `while` **loop:** Repeats as long as a condition is true.
- `do...while` **loop:** Similar to `while`, but guarantees the code runs at least once.

 Critical Warning: Loops that run an unpredictable or very high number of times can consume all available gas, causing the transaction to **fail**. They must be used carefully.

□ Functions Overview

Functions are the building blocks that define what a contract can *do*.

□ 1. Function Visibility Modifiers
These control who can call a function.

□ Modifier	□ Who Can Call?	□ Real-World Analogy
`public`	Anyone (inside or outside the contract)	A **public park**—open to everyone.
`private`	Only from inside the **same** contract	Your **diary**—only you can read it.
`internal`	From this contract and **inherited** contracts	A **company's staff room**—employees and their managers can enter.
`external`	Only from **outside** the contract	A **public phone booth**—only people from outside can use it.

□ 2. `view` and `pure` Functions (Gas Savers)
These functions promise not to modify the blockchain state.

□ Type	□ Can Read State?	□ Can Modify State?	□ Use Case
`view`	□ Yes	□ No	Getting a user's balance or a contract's settings.
`pure`	□ No	□ No	Performing a calculation, like adding two numbers.

- **Key Benefit:** When called externally, they don't cost gas.

 3. Return Values

- Functions can return data. They can return a single value or multiple values.
- Example: A function could return both a `username` (string) and a `score` (uint).

 4. Function Parameters

- Inputs you pass into a function. Example: A `greetUser` function would take a `name` (string) as a parameter.

 5. Constructors

- A **special function** that runs **only once** when the contract is deployed.
- Used for initial setup, like setting the contract's owner.

 6. Special Functions: `receive()` and `fallback()`

- `receive()`**:** A special function that is automatically executed when the contract receives **plain Ether** (with no data).
- `fallback()`**:** A catch-all function that runs when someone calls a function that doesn't exist in the contract or sends data along with Ether.

 7. Function Overloading & Overriding

- **Overloading:** Having multiple functions with the **same name** but **different parameters**. The correct one is called based on the inputs.
- **Overriding:** A child contract (from inheritance) can provide its own implementation of a function defined in a parent contract.

 Events & `emit` (Logging)

 What are Events?
Events are a way for a smart contract to **log information** onto the blockchain. This logged data is not accessible by other smart contracts but can be read by off-chain applications (like your website's frontend).

 How are they used?

- You **declare** an event in the contract.
- You `emit` the event inside a function when something important happens.
- Frontend apps (using Web3.js or Ethers.js) can **listen** for these events and update the UI in real-time.

  Example Use Case:
  A bank contract emits a `Transfer` event every time money is sent. A website can listen for this event and immediately update the user's displayed balance without them needing to refresh the page.

  Indexed Parameters

- You can mark up to three parameters in an event as `indexed`.
- This makes them **searchable and filterable**. It's like adding a hashtag to a social media post, allowing you to easily find all posts with that tag later.
- Example: Indexing the `from` and `to` addresses in a `Transfer` event lets you efficiently find all transfers to or from a specific wallet.

If-Else Statements

Introduction
If-Else statements allow a contract to make **decisions** and execute different code paths based on conditions.

- **Syntax:** `if (condition) { ... } else { ... }`
- The condition must evaluate to a boolean (`true` or `false`).
- Used for controlling the logic flow within functions.

  Example:
  A function `checkEligibility` could use an if-else statement to return "Eligible to vote" if `age >= 18`, and "Not eligible" otherwise.

---

7. Error Handling in Solidity

Solidity uses three main keywords to handle errors and ensure contract safety.

Feature	`require`	`assert`	`revert`
Purpose	**Validate inputs and conditions**	**Check for internal bugs** (should never fail)	**Trigger errors in complex logic**
Use Case	Is the user the owner? Is the amount > 0?	Checking for impossible states (e.g., math overflow)	Custom error handling inside `if-else` blocks
Gas Refund	✅ Yes (refunds unused gas)	❌ No (consumes all gas)	✅ Yes (refunds unused gas)
Analogy	**Bouncer at a club** checking your ID.	**Fire alarm**—it should only go off if something is seriously wrong.	**Pulling the emergency brake** on a train.

✅ **Example of `require`:**
`require(msg.sender == owner, "You are not the owner!");`
// If the condition is false, the transaction reverts with the message.

✅ Get and Set Functions

These are a fundamental pattern for interacting with a contract's data.

- `get` **Function:** A function (usually `view`) that **reads** a state variable and returns its value. It does not cost gas when called externally.
- `set` **Function:** A function that **writes** a new value to a state variable. This modifies the blockchain and **costs gas**.

✅ **Analogy:**
A safe deposit box.

- `get` is like looking through the box's window to see what's inside.
- `set` is like unlocking the box with a key and placing a new item inside.

□ Week 6 - Smart Contracts (Part 2: Deployment, Gas & Best Practices)

□ Smart Contract Deployment Lifecycle

1. **Source Code:** You write your contract in a `.sol` file.
2. **Compilation:** The Solidity compiler converts your human-readable code into two things:
   - **Bytecode:** Low-level machine code for the Ethereum Virtual Machine (EVM). This is what gets deployed to the blockchain.
   - **ABI (Application Binary Interface):** A JSON file that describes the contract's interface (function names, parameters, etc.). It acts as a manual for off-chain code to interact with the deployed contract.
3. **Deployment Transaction:** You send a transaction to the blockchain with the bytecode in the data field. This transaction has no `to` address.
4. **Contract Address:** The blockchain processes this transaction, runs the constructor, and generates a unique, permanent address for your contract.
5. **Interaction:** You and others can now interact with the contract at its new address using the ABI.

□ Understanding Gas, Execution Cost, and Transaction Cost

- **1□□ Gas:** The **unit of computational work** on the Ethereum network. Every operation (add, multiply, store data) costs a certain amount of gas. Think of it as the **fuel** or **effort** required.
- **2□□ Gas Price:** The amount of **Gwei** (a subunit of ETH) you are willing to pay per unit of gas. It's like the **price per liter** of fuel. You can set this higher to get your transaction processed faster.
- **3□□ Execution Cost:** The total gas used by your contract's logic **multiplied by the gas price**. This is the fee for the computation itself.
  - `Execution Cost = Gas Used * Gas Price`
- **4□□ Transaction Cost:** The total cost of sending the transaction. This includes the **execution cost** plus a **base fee** that every transaction must pay.
  - `Transaction Cost = (Gas Used + 21,000) * Gas Price`

Simple Analogy:

- **Gas:** Liters of fuel needed for a car trip.
- **Gas Price:** Cost per liter at the pump.
- **Execution Cost:** Total fuel cost for the trip itself.
- **Transaction Cost:** Total fuel cost + a fixed toll fee for using the highway.

Hands-On: Remix Deployments & Gas Observations

Tools:

- **Remix IDE:** An online tool for writing, compiling, and deploying smart contracts.
- **Environments in Remix:**
  - **JavaScript VM:** A simulated blockchain in your browser. Perfect for fast testing and debugging. **No real ETH needed.**
  - **Injected Provider (e.g., MetaMask):** Connects Remix to your MetaMask wallet, allowing you to deploy to **real testnets** (like Sepolia or Goerli). Requires testnet ETH.

What to Observe:

- **Deploying a Contract:** This is a single, large transaction that consumes a significant amount of gas.
- **Calling a `view`/`pure` Function:** Costs **no gas** because it only reads data and doesn't change the blockchain state.
- **Calling a State-Changing Function:** (e.g., a `set` function) costs **gas** because it requires miners/validators to process and store the new data.

Contract ABI & Bytecode

- **Bytecode:** This is the **compiled contract** in a form the EVM can understand. It's what is actually stored on the blockchain. You can think of it as the **machine code** of your smart contract.

- **ABI (Application Binary Interface):** This is a **JSON file** that describes how to interact with the contract. It lists all the functions, their inputs and outputs, and the events. It's the **user manual** that your frontend application needs to talk to your deployed contract.

  How they work together:

1. The **Bytecode** is deployed to create the live contract on-chain.
2. Your web app uses the **ABI** and the contract's **address** to create a JavaScript object that can call the functions on the live contract.

Week 6 - Class 4: Modifiers, Payments & Mini Project

What Are Modifiers?

Modifiers are reusable pieces of code that you can attach to functions to **automatically check a condition before the function runs**.

- **Syntax:** `modifier onlyOwner() { require(msg.sender == owner, "Not owner!"); _; }`
- The `_;` symbol indicates where the original function's code will be inserted.

Common Use Cases:

- `onlyOwner`**:** To restrict a function so that only the contract administrator can call it.
- `notPaused`**:** To temporarily disable certain functions in case of an emergency.
- `validAmount`**:** To check that an input value is greater than zero, preventing invalid transactions.

Payable Functions & Ether Transfers

The `payable` Keyword

- A function or address must be marked as `payable` to be able to **receive Ether**.
- If you try to send ETH to a non-payable function, the transaction will be rejected.

□ Sending Ether from a Contract

There are three primary ways for a contract to send ETH to another address:

Method	Auto-Revert on Failure?	Gas Limit	Recommendation
`transfer`	□ Yes	2300 gas	Simple, but limited.
`send`	□ No	2300 gas	Unsafe; you must manually check the result.
`call`	□ No	Forwards all gas	**Current best practice.** More flexible and powerful.

□ **Example:** A `withdraw` function in a contract would use one of these methods to send the contract's balance to the owner's address.

□ Creating Your Own Cryptocurrency (ERC-20 Token)

□ What is ERC-20?

ERC-20 is a **technical standard** for fungible tokens on Ethereum. It defines a common set of rules that all Ethereum tokens must follow, ensuring they can work seamlessly with wallets, exchanges, and other smart contracts.

□ Core Features of an ERC-20 Token:

1. `totalSupply()`**:** Returns the total number of tokens in existence.
2. `balanceOf(address)`**:** Returns the token balance of a specific account.
3. `transfer(to, amount)`**:** Allows the caller to send tokens to another address.
4. `approve(spender, amount)`**:** Allows a `spender` (like a DEX) to withdraw a certain amount from your account.
5. `transferFrom(from, to, amount)`**:** Allows the approved spender to transfer tokens on your behalf.
6. `allowance(owner, spender)`**:** Returns the remaining amount that a spender is allowed to withdraw from the owner.

Common Extra Features:

- `mint`**:** The ability to create new tokens (usually restricted to the owner).
- `burn`**:** The ability for users to destroy their own tokens, reducing the total supply.
- `pause`**/**`unpause`**:** An emergency stop function to halt all token transfers.

The `decimals` Property:

- Most tokens, like ETH, are divisible by 18 decimal places.
- This means 1 full token is represented as `1000000000000000000` in the contract's internal storage.
- It allows for micro-transactions, like sending 0.001 tokens.

Non-Fungible Tokens (NFTs)

Definition
An NFT (Non-Fungible Token) is a **unique digital asset** on the blockchain that represents ownership of a specific item. Unlike ERC-20 tokens, where every token is identical, each NFT is one-of-a-kind.

Fungible vs. Non-Fungible

- **Fungible (ERC-20):** Interchangeable. Your 1 ETH is identical to my 1 ETH.
- **Non-Fungible (ERC-721):** Unique. Your CryptoPunk #1234 is different from every other CryptoPunk.

Metadata: The "Soul" of an NFT

- The NFT on the blockchain is essentially just a unique ID.
- The **metadata** is a JSON file that describes the NFT: its **name, description, image URL, and attributes** (e.g., "Rarity: Legendary").
- This metadata is typically stored on **IPFS** (a decentralized file storage system) to ensure it is permanent and tamper-proof.

NFT Lifecycle:

1. **Deploy Contract:** An ERC-721 smart contract is deployed.

2. **Mint NFT:** The contract's `mint` function is called, which creates a new token with a unique ID and assigns it to an owner. The function links the token ID to its metadata URI.
3. **View NFT:** Wallets (MetaMask) and marketplaces (OpenSea) read the contract, see the token ID and its owner, fetch the metadata from IPFS, and display the image and details to the user.

Ethereum Token Standards

Standard	Purpose
ERC-20	The standard for **fungible tokens** (e.g., cryptocurrencies, stablecoins like USDC).
ERC-721	The standard for **Non-Fungible Tokens (NFTs)** (e.g., CryptoPunks, Bored Apes).
ERC-1155	A **multi-token standard** that can handle both fungible and non-fungible tokens in a single contract (common in gaming).

This concludes the detailed notes for your smart contract foundations course, covering all the theoretical concepts and their practical implications without the specific Solidity syntax.